



On the Verification Problem of Remote Direct Memory Access Programs

Parosh Aziz Abdulla^{1,2} , Mohamed Faouzi Atig¹ , Govind Rajanbabu³ ,
and Stephan Spengler¹

¹ Uppsala University, Uppsala, Sweden

{parosh.abdulla,mohamed_faouzi.atig,stephan.spengler}@it.uu.se

² Mälardalen University, Västerås, Sweden

³ The Institute of Mathematical Sciences (IMSc), Chennai, India
govind@imsc.res.in

Abstract. Remote Direct Memory Access (RDMA) is a technology that allows direct memory access from the memory of one computer into that of another without involving either one's operating system. This enables high-throughput, low-latency networking, which is especially useful in massively parallel computer clusters.

In this paper, we study the reachability and robustness problems for RDMA programs. We show that reachability is undecidable in general, even for a restricted fragment of the model. We then focus on robustness, which asks whether a program exhibits the same behaviours under the RDMA and sequential consistency (SC) semantics, and prove that this problem is decidable. Our central technical result establishes a normal form for robustness violations, showing that any non-robust program admits a violating execution of a specific form. We then leverage this normal form to obtain a decision procedure that reduces robustness to reachability in finite-state programs with counters, yielding an EXPSpace upper bound in the general case, and a PSPACE upper bound in the absence of poll operations. Finally, we also show that both of these bounds are optimal.

Keywords: RDMA · Weak Memory Models · Robustness · Verification · Concurrent Systems

1 Introduction

Modern multiprocessor architectures implement weak memory models that relax the guarantees of sequential consistency (SC) to enable aggressive hardware and compiler optimisations. Classical models such as TSO, PSO, ARM, POWER, and RC11 permit various forms of reordering, buffering, and speculation, thereby admitting behaviours that are impossible under SC. While essential for performance, these relaxations significantly complicate reasoning about concurrent programs. This increased complexity is inherent and well-established. For

instance, reachability for finite-state concurrent programs is PSPACE-complete under SC, but becomes non-primitive recursive under TSO and undecidable under the Release-Acquire (RA) fragment of RC11. Thus, verifying safety properties under weak memory models is fundamentally harder than under SC, posing significant challenges for program analysis.

Remote Direct Memory Access (RDMA) has recently gained prominence in high-performance computing [10, 11]. RDMA enables a process to directly read from or write to the memory of a remote machine, bypassing the remote CPU and operating system on the critical path. This paradigm is widely used in data centres, where it offers low latency and high throughput. However, RDMA introduces a non-trivial memory model that differs fundamentally from both shared-memory concurrency and message passing. RDMA supports one-sided operations, provides weak ordering guarantees, and allows remote memory updates to be observed asynchronously with respect to local CPU accesses. Consequently, reasoning about correctness requires explicit consideration of visibility, ordering, and synchronisation mechanisms such as fences and completion events. These characteristics distinguish RDMA from existing weak memory models, necessitating specialised verification techniques.

In this work, we first study the reachability problem for RDMA programs. We show that reachability is undecidable in general, via a reduction from Post's Correspondence Problem. Notably, this undecidability holds even for finite-state RDMA programs without *poll* operations or *fence* instructions, highlighting the inherent complexity of the model.

The difficulty of proving safety properties under weak memory has motivated the study of correctness notions that enable sound reasoning despite relaxed semantics. A fundamental question is whether a program behaves *robustly* under a given weak memory model [14]. Informally, a program is robust if every behaviour under the weak model is observationally equivalent to some behaviour under SC. Robustness is appealing for two reasons. First, if a program is robust and correct under SC, then it remains correct under the weak model. Second, verifying robustness can be simpler than verifying safety directly. For example, checking reachability under TSO is non-primitive-recursive [12, 13], whereas checking robustness is PSPACE-complete [14].

Recently, Ambal et al. [11] initiated the study of robustness for RDMA programs. They established sufficient conditions for robustness, but these are restrictive and do not characterise all robust programs (see, e.g., Fig. 6(b) in [11]). Their work leaves open the decidability and complexity of robustness. In this work, we resolve these questions by showing that robustness of finite-state RDMA programs is decidable and EXSPACE-complete. Our approach relies on a structural characterisation of robustness violations, enabling a reduction to reachability in SC programs augmented with finite control and counters.

To formalise RDMA behaviours, we build on the axiomatic semantics of Ambal et al. [10]. Their consistency check requires verifying acyclicity of two relations over execution events, which is rather complex. We revisit this semantics and introduce a simpler, equivalent characterisation of RDMA consistency.

This refined view serves as the foundation for our decidability results. To establish the EXPSPACE upper bound, we introduce a novel instrumentation technique that transforms an RDMA program into an SC program. This SC program simulates all behaviours of the original while preserving observational equivalence, enabling the use of existing SC verification tools. We also identify a simpler fragment – RDMA programs without poll operations – for which robustness reduces to reachability in purely finite-state SC programs.

In summary, our main contributions are:

- We prove that reachability for RDMA programs is undecidable, even without poll operations or fences.
- We simplify the axiomatic RDMA semantics of [10], providing an equivalent but more tractable consistency check.
- We establish a normal form for robustness violations by rearranging computations around a pivotal event.
- We present a decision procedure for robustness based on an instrumentation that transforms RDMA programs into SC programs with counters, yielding an EXPSPACE upper bound (PSPACE without poll operations).
- We prove matching lower bounds: robustness is EXPSPACE-complete in general and PSPACE-complete without poll operations.

Related Work. We base our formalisation of RDMA behaviours on the RDMA semantics proposed by Ambal et al. [10,11]. In [10], the authors assume TSO processors per node, while [11] considers SC processors. Both works present axiomatic and operational semantics for RDMA and propose consistency-checking procedures. An earlier semantics by Dan et al. [19] fails to capture certain RDMA behaviours, as shown in [10].

The reachability problem has been extensively studied for various weak memory models, such as TSO [5,8,12,13], RA [1], Causal Consistency [23], PSO [7], Power [2], Intel-x86 with Persistency [3,4], Promising Semantics [6], and Localized Release-Acquire [27]. The complexity of the reachability problem varies significantly across these models, ranging from non-primitive recursive to undecidable, depending on the specific memory model and program restrictions. To the best of our knowledge, our work is the first to study the reachability problem for RDMA programs, establishing its undecidability in general.

The robustness problem has also been extensively studied for various memory models [14–17,20,21]. For RDMA, Ambal et al. [11] initiated the study of robustness, providing sufficient (but restrictive) conditions for ensuring robustness; their paper also gives robust examples not covered by these conditions (Fig. 6(b) in [11]). To the best of our knowledge, our work is the first to establish decidability and complexity results for robustness and reachability-based safety verification under RDMA semantics.

2 RDMA Program Semantics

A program consists of a finite set of threads that operate on a distributed memory system composed of multiple nodes. Each node contains its own local mem-

ory and processes, and nodes communicate with each other using Remote Direct Memory Access (RDMA) operations. We consider a finite instruction set consisting of local reads and writes, remote write operations, remote read operations, polling instructions, and a remote-fence instruction. Local instructions operate directly on the node's memory, whereas remote instructions access the memory of other nodes via their *network interface controllers* (NIC).

In the following, $x, y, z \in \text{Loc}$ denote memory locations, $v_r, v_w \in \text{Val}$ denote values, and $n \in \text{Node} = \{1, \dots, N\}$ denotes nodes. A bar above a location or node indicates a remote location or node, respectively. Each thread (or *process*) P in an RDMA program is defined by the following grammar:

$$\begin{aligned} P ::= & \text{skip} \mid P_1; P_2 \mid P_1 + P_2 \mid P^* \\ & \mid x := v_w \mid \text{assume}(x = v_r) \mid \text{assume}(x \neq v_r) \mid x := \text{CAS}(z, v_r, v_w) \\ & \mid x := \bar{y} \mid \bar{y} := x \mid \text{rfence}(\bar{n}) \mid \text{poll}(\bar{n}) \end{aligned}$$

To model communication between nodes, each process P maintains a distinct buffer system (or *queue pair* in the terminology of [10]) for every remote node $\bar{n}$ whose memory it may access. Remote operations issued by process P towards node $\bar{n}$ are first placed in the buffer system, which consists of multiple FIFO buffers. Their arguments are not defined at the point when they are issued, since they may depend on local memory reads that occur after the remote operation is issued. Furthermore, they need to travel through the buffer system before they are propagated to the remote node. Therefore, the completion of remote operations is not instantaneous, and may occur out of order with respect to the program order of the issuing process. This allows the reordering of remote operations with respect to each other and with respect to local memory accesses, inducing a weak memory model.

We briefly describe the key buffers in the queue system; the full operational model involves additional buffers and is detailed in [10]. When a process issues a remote write, a symbolic entry $\bar{y} := x$ is placed in the first buffer B_1 . This entry is not yet instantiated with the value of x – the local variable x may be modified by subsequent CPU operations before the entry is processed. When the entry reaches the head of B_1 , the current value v of x is read from local memory, and an instantiated entry $\bar{y} := v$ is forwarded to buffer B_2 . This delayed instantiation allows local writes to overtake the value read by pending remote operations. Buffer B_2 transmits the instantiated write to the remote node, where it updates the target memory location $\bar{y}$. Upon completion, an acknowledgment is placed in buffer B_3 , which returns to the issuing node. Remote reads follow a similar pattern, with instantiation occurring at the remote node. Both remote read and remote write operations generate acknowledgments that can be consumed by `poll` operations.

`poll` operations provide a mechanism for synchronising with the completion of remote operations. A poll instruction `poll( $\bar{n}$ )` blocks until the earliest (in program order) pending remote operation issued by the calling process towards node $\bar{n}$ has completed. `poll` operations can be used to prevent undesired reordering between remote operations and subsequent local memory accesses by ensuring

that a remote operation has completed before execution proceeds. However, note that each `poll` waits for exactly one remote operation to complete, rather than for all pending operations towards the same node. A remote-fence instruction `rfence` ensures that all remote operations issued by the calling process towards a node prior to the fence are completed before any subsequent remote operations towards the same node are issued.

We will recall the full details of the axiomatic semantics in Sect. 4.

3 Undecidability of Reachability

In this section, we show that the reachability problem for RDMA programs is undecidable. Here, reachability is classical control-state reachability: given an RDMA program and a control state of one thread, we ask whether some execution from the initial configuration reaches a configuration in which that thread is in the target control state, while all other threads may be in arbitrary control states. The proof will follow from a reduction from Post's Correspondence Problem (PCP) [25], a well-known undecidable problem. An instance of PCP is given by two sequences of non-empty words over some finite alphabet Σ , say $u_1, u_2, \dots, u_N$ and $v_1, v_2, \dots, v_N$. The problem asks whether there exists a sequence $i_1, i_2, \dots, i_k$ of indices such that $u_{i_1}u_{i_2} \dots u_{i_k} = v_{i_1}v_{i_2} \dots v_{i_k}$. We will propose an RDMA program R consisting of four processes P_1, P_2, P_3 , and P_4 , each on a different node, such that there is an execution of R reaching a final configuration (i.e., all four processes terminate) if and only if PCP has a solution for the aforementioned instance (Fig. 1).

The idea is that process P_1 guesses a solution of PCP, which is a sequence of indices $i_1, i_2, \dots, i_k$, and sends the guessed indices to both P_2 and P_3 . For each received index i , process P_2 (P_3) sends the letters of u_i (v_i) to process P_4 . Process P_4 checks that the sequences of letters received from P_2 and P_3 are identical.

In order to implement this idea, we need to ensure that the sequences of letters sent by the processes are received without any losses or duplications. Since the set of letters is finite, the problem amounts to showing that we can transfer a sequence of letters over a finite alphabet from one process to another. The main challenge is that remote writes propagate through the buffer system (cf. Sect. 2), and the receiving process has no built-in mechanism to detect when a new value arrives. This can lead to missed or duplicated reads: the receiver may read before the value arrives, or read the same value multiple times. We address this using an alternating bit protocol.

Figure 2 illustrates the problem. Process P_1 wants to send a sequence of symbols from a finite alphabet $\{1, 2\}$ to process P_2 . To send each symbol, process P_1 uses a dedicated register for that symbol (register a always holds 1, register

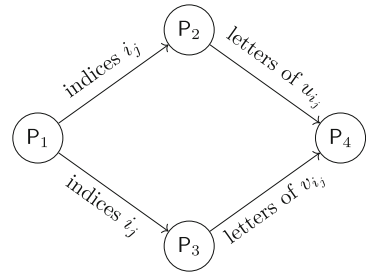


Fig. 1. The PCP program.

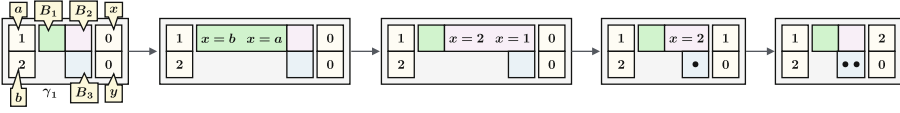


Fig. 2. Message loss due to unsynchronised reads.

b always holds 2), and performs $\bar{x} := a$ or $\bar{x} := b$ accordingly. Using constant values avoids complications from the delayed instantiation in buffer B_1 (see Sect. 2). However, process P_2 may not observe all sent symbols: in Fig. 2, process P_1 sends the sequence 1, 2, but P_2 may read only the final value 2, missing the intermediate 1. To fix this synchronisation problem, we use an alternating bit protocol (ABP) between each pair of processes. The ABP ensures that each sent symbol is received exactly once by the receiving process. The protocol uses two extra registers f and g at the sender and receiver processes, respectively. Register f at the sender process holds the alternating bit, while register g at the receiver process observes the alternating bit. The key invariant is that each symbol is accepted only when g changes to the sender's bit, which prevents both losses and duplicate deliveries. The protocol works as follows (see Fig. 3). The initial value of f and g are $\top$ and $\perp$, respectively, while a and b hold the values from the alphabet (here 1 and 2). In Fig. 3, we assume that process P_1 wants to send a word over the alphabet $\{1, 2\}$ to process P_2 , of the form $\dots 21$, i.e., the word starts with 1 followed by 2. The process P_1 first performs $\bar{x} := a$ to send the value 1 to P_2 , and then $\bar{g} := f$ to set the flag g to $\top$, indicating that a new value 1 has been sent (see configuration γ_2). When these two messages reach the head of buffer B_1 , they acquire their values (1 resp. $\top$), and are forwarded to buffer B_2 (cf. configuration γ_3). When $x = 1$ is at the head of buffer B_2 , the node of P_2 updates its local variable x to 1 (cf. configuration γ_4). Once process P_2 has read the value 1, we need to ensure that it can detect and read the next value. To that end, process P_2 carries out a sequence of operations, where it first performs the local write operation $\text{wr}(x, \perp)$ to reset x to its initial value $\perp$. Then it checks that the receiver-side flag is still unset by performing the local read operation $\text{rd}(g, \perp)$ (cf. configuration γ_5). This ensures that no additional write operations were performed in the previous steps. Next, process P_2 performs the local read operation $\text{rd}(g, \top)$ to detect the next fresh delivery and ensure that the old value 1 will not be read again (cf. configuration γ_8). Finally, it resets the flag g to its initial state $\perp$.

Using this protocol, we can implement the PCP reduction as follows (details can be found in the extended version [9]). Process P_1 uses the ABP to send the guessed indices to both P_2 and P_3 . For each received index i , process P_2 uses the ABP to send the letters of u_i to P_4 , while P_3 sends the letters of v_i . Process P_4 checks that the two received letter sequences are identical, alternating between receiving from P_2 and P_3 . The buffered communication is essential: since P_2 and P_3 may produce letters at different rates, the buffers decouple their execution, allowing the final sequences to match even when the corresponding words u_i and

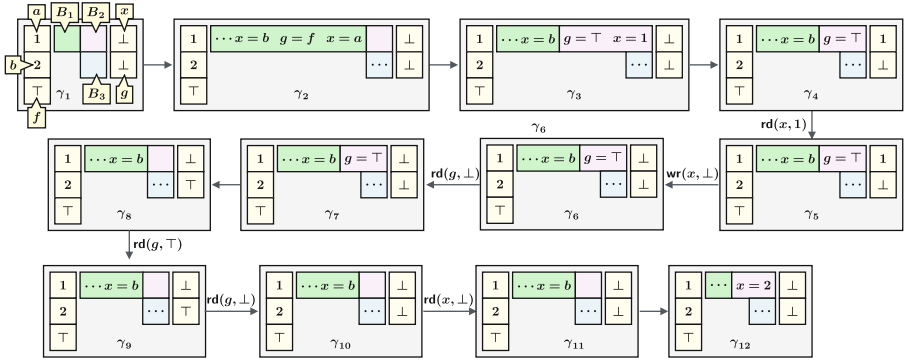


Fig. 3. The Alternating Bit Protocol.

v_i have different lengths. The program reaches a final configuration if and only if PCP has a solution. This establishes the following result.

Theorem 1. *The reachability problem for RDMA programs is undecidable.*

We note that the reduction uses only a restricted fragment of the RDMA model: it requires only local operations and remote writes, without remote reads or any synchronisation primitives such as `poll` or `rfence`. This demonstrates that reachability is undecidable even for this minimal fragment.

4 The Declarative Semantics of RDMA

We build upon the declarative semantics for RDMA developed by Ambal et al. [10, 11]. The CPU part of our model and the general notion of sequential consistency follow standard definitions from the literature. The RDMA-specific aspects – NIC operations, their labels and events, `pf`, `nfo` and derived relations, and the RDMA consistency conditions – are adopted from their work.

An RDMA program R is a collection of threads $(P_t)_{t \in \text{Tid}}$, where $\text{Tid} = \{1, \dots, M\}$ is the set of thread identifiers. We assume a function $n : \text{Loc} \rightarrow \text{Node}$ that maps variables to their node. An execution of a program generates events, which are labelled by their type. Let Lab consist of all labels of the following forms:

NIC remote read:	$\text{nrR}(\bar{y}, v_r)$	CPU local read:	$\text{1R}(x, v_r)$
NIC local write:	$\text{n1W}(x, v_w, \bar{n})$	CPU local write:	$\text{1W}(x, v_w)$
NIC local read:	$\text{n1R}(x, v_r, \bar{n})$	CPU compare-and-swap:	$\text{CAS}(z, v_r, v_w)$
NIC remote write:	$\text{nrW}(\bar{y}, v_w)$	CPU poll:	$\text{poll}(\bar{n})$
NIC fence:	$\text{nF}(\bar{n})$		

An *event* is defined as a triple $e = \langle \iota, t, l \rangle$ where $\iota \in \mathbb{N}$ is the program counter, $t \in \text{Tid}$ is the thread id, and $l \in \text{Lab}$ is a label. We omit the program counter

ι and the thread id t when they are clear from the context, i.e. the event is identified by its label. Some operations generate multiple events. A remote read operation $x := \bar{y}$ generates a NIC remote read $\text{nrR}(\bar{y}, v)$ followed by a NIC local write $\text{nlW}(x, v, n(\bar{y}))$. A remote write operation $\bar{y} := x$ works similarly: it generates a NIC local read $\text{nlR}(x, v, n(\bar{y}))$ followed by a NIC remote write $\text{nrW}(\bar{y}, v)$. A compare-and-swap operation $x := \text{CAS}(z, v_r, v_w)$ generates either (on success) $\text{CAS}(z, v_r, v_w) \cdot \text{1W}(x, v_r)$ or (on failure) $\text{1R}(z, v) \cdot \text{1W}(x, v)$ where $v \neq v_r$. We inductively define $\text{Seq}(\mathbf{P}) \subseteq \text{Lab}^*$ as the set of all finite label sequences that can be generated by the process $\mathbf{P}$. The formal definition is standard and therefore omitted, but can be found in the extended version [9].

Notation. We use $\iota(e)$, $t(e)$, $n(e)$, $\bar{n}(e)$, $\text{loc}(e)$, $v_r(e)$ and $v_w(e)$ to refer to the program counter, thread id, local node, remote node, location, read value and write value of e , whenever applicable.

Let E be a set of events. With $E.\text{nrR}$, $E.\text{nlW}$, $\dots$ we refer to the subset of events of E that have label nrR , nlW , $\dots$, respectively. We define:

$$\begin{aligned} E.\text{CPU} &= E.\text{1R} \cup E.\text{1W} \cup E.\text{CAS} \cup E.\text{poll} && (\text{CPU events}) \\ E.\text{nW} &= E.\text{nlW} \cup E.\text{nrW} && (\text{NIC write events}) \\ E.\text{W} &= E.\text{nlW} \cup E.\text{nrW} \cup E.\text{1W} \cup E.\text{CAS} && (\text{write events}) \\ E.\text{R} &= E.\text{nlR} \cup E.\text{nrR} \cup E.\text{1R} \cup E.\text{CAS} && (\text{read events}) \\ E.\text{Inst} &= E \setminus (E.\text{nlW} \cup E.\text{nrW}) && (\text{instantaneous events}) \end{aligned}$$

Given a relation $\text{rel} \subset E \times E$ and a subset $E' \subset E$, we use $\text{rel}^{-1} = \{(e_2, e_1) \mid (e_1, e_2) \in \text{rel}\}$, $[E'] = \{(e, e) \mid e \in E'\}$ and $\text{rel} \upharpoonright_{E'} = \text{rel} \cap E' \times E'$. We usually write $e_1 \xrightarrow{\text{rel}} e_2$ if $(e_1, e_2) \in \text{rel}$. If $\text{rel}' \subset E \times E$ is another relation, then $\text{rel}; \text{rel}' = \{(e_1, e_3) \mid \exists e_2 \in E : (e_1, e_2) \in \text{rel}, (e_2, e_3) \in \text{rel}'\}$. Lastly, we define the *same-queue-pair* relation $\text{sqp} = \{(e_1, e_2) \in E \times E \mid t(e_1) = t(e_2) \wedge \bar{n}(e_1) = \bar{n}(e_2)\}$.

Definition 2 (Trace). A trace of an RDMA program $\mathbf{R} = (\mathbf{P}_t)_{t \in \text{Tid}}$ is a tuple $T = \langle E, \text{po}, \text{pf}, \text{rf}, \text{mo}, \text{nfo} \rangle$ where:

- $E = E^0 \cup \bigcup_{t \in \text{Tid}} E^t$ is a finite set of events such that:
 - $E^0 = \{(-, -, \text{1W}(x, v_x)) \mid x \in \text{Loc}\}$ is the set of initialisation events, where v_x is the initial value of x .
 - For each $t \in \text{Tid}$, there is $s_t \in \text{Seq}(\mathbf{P}_t)$ and $0 \leq n \leq |s_t|$ such that $E^t = \{\langle \iota, t, s_t[\iota] \rangle \mid 1 \leq \iota \leq n\}$ (i.e., the events of each thread form a prefix of some path in $\text{Seq}(\mathbf{P}_t)$).
- $\text{po} \subset \bigcup_{t \in \text{Tid}} (E^t \times E^t)$ is the program order relation on E , which is a strict total order on the events of each thread, uniquely defined such that $(e_1, e_2) \in \text{po}$ if and only if $\iota(e_1) < \iota(e_2)$ and $t(e_1) = t(e_2)$.
- $\text{pf} \subset (E.\text{nW} \times E.\text{poll}) \cap \text{po} \cap \text{sqp}$ is the polls-from relation on E , that relates NIC write events to later (w.r.t. po) poll events on the same queue pair. It is uniquely defined such that every poll event polls from exactly one NIC write,

which must be the earliest (w.r.t. po) non-pollled one. That is, $(e_1, e_2) \in \text{pf}$ if and only if $e_1 \in E.\text{nW}$, $e_2 \in E.\text{poll}$, $(e_1, e_2) \in \text{po} \cap \text{sqp}$ and:

$$|\{e'_1 \in E.\text{nW} \mid (e'_1, e_1) \in \text{po} \cap \text{sqp}\}| = |\{e'_2 \in E.\text{poll} \mid (e'_2, e_2) \in \text{po} \cap \text{sqp}\}|$$

- $\text{rf} \subset E.\text{W} \times E.\text{R}$ is the **reads-from** relation on E , which relates write events to read events on the same variable and with the same value. That is, $(e_1, e_2) \in \text{rf}$ only if $\text{loc}(e_1) = \text{loc}(e_2)$ and $v_w(e_1) = v_r(e_2)$. Furthermore, each read event reads from exactly one write event.
- $\text{mo} = \bigcup_{x \in \text{Loc}} \text{mo}_x \subset E.\text{W} \times E.\text{W}$ is the **modification order** relation on E . Each mo_x is a strict total order on the write events on location x , such that $E_x^0 \times (E.\text{W}_x \setminus E_x^0) \subset \text{mo}_x$, where $E_x^0 = \{(-, -, \text{IW}(x, v_x))\} \subseteq E^0$ and $E.\text{W}_x = \{e \in E.\text{W} \mid \text{loc}(e) = x\}$.
- $\text{nfo} \subset \text{sqp}$ is the **NIC flush order** relation on E , which relates NIC events on the same queue pair in the following way: For all $(e_1, e_2) \in \text{sqp}$, if $e_1 \in E.\text{nIW}$ and $e_2 \in E.\text{nIW}$, then exactly one of the pairs (e_1, e_2) or (e_2, e_1) is in nfo . The same holds true for $e_1 \in E.\text{nrR}$ and $e_2 \in E.\text{nrW}$.

Definition 3 (Derived Relations). Given a trace $T = \langle E, \text{po}, \text{pf}, \text{rf}, \text{mo}, \text{nfo} \rangle$, we define:

$$\begin{aligned} \text{rb} &= (\text{rf}^{-1}; \text{mo}) && (\text{reads-before}) \\ \text{sc} &= (\text{po} \cup \text{rf} \cup \text{rb} \cup \text{mo})^+ && (\text{sequential consistency}) \\ \text{ippo} &\subset \text{po} \text{ as in Figure 4} && (\text{issue-preserved program order}) \\ \text{oppo} &\subset \text{po} \text{ as in Figure 4} && (\text{observation-preserved program order}) \\ \text{ib} &= (\text{ippo} \cup \text{rf} \cup \text{pf} \cup \text{nfo})^+ && (\text{issued-before}) \\ \text{ob} &= (\text{oppo} \cup \text{rf} \cup ([\text{nIW}]; \text{pf}) \cup \text{nfo} \cup \text{rb} \cup \text{mo} \cup ([\text{Inst}]; \text{ib}))^+ && (\text{observed-before}) \end{aligned}$$

The rb relation ensures that if $w_1 \xrightarrow{\text{rf}} r$ and $w_1 \xrightarrow{\text{mo}} w_2$, then r was observed before w_2 . To build intuition for the remaining relations, recall that NIC writes are non-instantaneous (they may be buffered), while events in Inst are instantaneous. Hence we distinguish *issue* order from *observation* order.

The relation ippo captures po -edges that must be preserved at issue points, whereas oppo captures those that must be preserved at observation points. Thus oppo is closer to the usual notion of preserved program order. In Fig. 4, a $\checkmark$ means that po -edges between the two event types are contained in both ippo and oppo , whereas $\times$ means that po -edges are not contained in either relation. sqp means that po -edges between two events on the same queue pair are preserved in both relations and $\text{ippo} : \text{sqp}$ means that these edges are only preserved in ippo but not oppo . For example, a NIC write followed by a fence on the same queue pair is in ippo but not necessarily in oppo : the write is issued first, but may be observed later due to buffering.

$po \hookrightarrow$	CPU	nlR	nrW	nrR	nlW	nF
CPU	✓	✓	✓	✓	✓	✓
nlR	×	sqp	sqp	sqp	sqp	sqp
nrW	×	×	sqp	sqp	sqp	ippo: sqp
nrR	×	×	×	×	sqp	sqp
nlW	×	×	×	×	sqp	ippo: sqp
nF	×	sqp	sqp	sqp	sqp	sqp

Fig. 4. The relations **ippo** and **oppo** as subsets of po .

The relation **ib** collects ordering constraints at issue points (including **ippo**, **rf**, **pf**, and **nfo**), while **ob** is the corresponding observation-level causality relation (including **oppo**, **rf**, $([nlW]; pf)$, **nfo**, **rb**, **mo**, and $([Inst]; ib)$). Intuitively, $([Inst]; ib)$ lifts issue-order constraints to observation order when the source event is instantaneous.

We use the notation ‘ T .’ to refer to the various components of T , e.g. $T.E$ is the event set of T and $T.ib$ is the **ib**-relation of T .

Definition 4 (Consistency and Robustness). *A trace T is called*

- SC-consistent *if $T.sc$ is acyclic (irreflexive).*
- RDMA-consistent [11] *if both $T.ib$ and $T.ob$ are acyclic (irreflexive).*

A program R is said to be robust if all its RDMA-consistent traces are also SC-consistent. We call a trace violating if it is RDMA-consistent but not SC-consistent.

The definition of SC consistency is standard. The definition of RDMA consistency is well motivated by Ambal et al. who show that the operational and declarative semantics of RDMA are equivalent. Our first result is that the first condition of RDMA-consistency, acyclicity of the issued-before relation, is redundant:

Theorem 5. *If **ib** is cyclic, then **ob** is cyclic.*

Proof. Consider a cycle in **ib**, i.e. a sequence of events $e_0, e_1, \dots, e_n = e_0$ such that $e_{i-1} \xrightarrow{rel_i} e_i$ and $rel_i \in \{\text{ippo}, \text{rf}, \text{pf}, \text{nfo}\}$ for all $1 \leq i \leq n$. If all edges of the cycle are in **ob**, we are done. So, assume that there exists at least one edge $e_{i-1} \xrightarrow{rel_i} e_i$ that is not contained in **ob**. In particular, it holds that:

$$\begin{aligned}
 (e_{i-1}, e_i) &\in (\text{ippo} \cup \text{rf} \cup \text{pf} \cup \text{nfo}) \setminus \text{ob} \subseteq (\text{ippo} \setminus \text{oppo}) \cup ([nrW]; \text{pf}) \\
 &= ([nrW]; po; [nF]) \cup ([nlW]; po; [nF]) \cup ([nrW]; \text{pf})
 \end{aligned}$$

The event e_i must be a NIC fence or a poll event, which are both instantaneous. We conclude $(e_i, e_i) \in \text{ib}$ and since $([Inst]; \text{ib}) \subseteq \text{ob}$, it follows that $(e_i, e_i) \in \text{ob}$, which means that **ib** is cyclic.

Henceforth, we only need to check acyclicity of **ob** when verifying RDMA-consistency.

5 Normal Form for Violating Computations

Our main result is that every non-robust RDMA program has a violating trace in a special *normal form*:

Theorem 6 (Normal Form). *If an RDMA program R has a violating trace, then it has a violating trace whose event set can be partitioned as $E = E_1 \cup E_2$ such that:*

1. *The events within E_1 and E_2 are in an SC-consistent order, respectively. That is, $\text{sc}|_{E_1}$ and $\text{sc}|_{E_2}$ are acyclic.*
2. *All **ob**-edges between E_1 and E_2 go from E_1 to E_2 , that is, for all $e_1 \in E_1$ and $e_2 \in E_2$, if e_1 and e_2 are related by **ob**, then $e_1 \xrightarrow{\text{ob}} e_2$.*

The key insight is that we can simplify any violating trace by identifying a *maximal event* – one that could be the last issued event – and reorganising the execution around it. This reorganisation removes unnecessary delays while preserving both RDMA-consistency and the SC-violation.

This chapter presents the construction of the normal form trace and establishes the correctness of Theorem 6. The next chapter shows how this normal form enables a practical robustness decision procedure: we can systematically search for violating traces by building up E_1 and E_2 in parallel during a single SC-simulation, non-deterministically assigning each generated event to either E_1 or E_2 .

5.1 Example

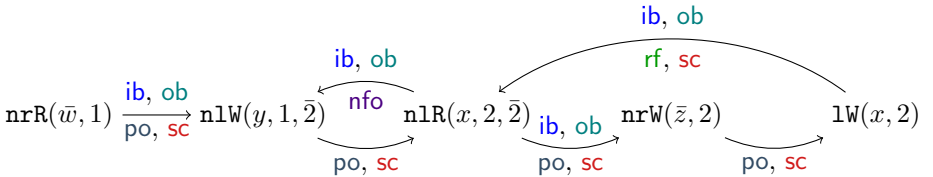
Consider the program R on the right. It has two nodes with two variables each: node 1 has variables x and y , node 2 has variables z and w . We consider only a single thread on node 1, with three instructions: a remote read of w into y , a remote write of x into z , and a local write of 2 into x . This program is not robust:

$x = 1$	$z = 0$
$y = 0$	$w = 1$
1: $y := \bar{w}$	
2: $\bar{z} := x$	
3: $x := 2$	

The read on x in line 2 can be delayed after the write on x in line 3 due to the RDMA memory model, causing it to read the value 2 instead of the initial value 1. This gives rise to a trace $T = \langle E, \text{po}, \text{pf}, \text{rf}, \text{mo}, \text{nfo} \rangle$ with event set

$$E = E_0 \cup \{\text{nrR}(\bar{w}, 1), \text{n1W}(y, 1, \bar{2}), \text{n1R}(x, 2, \bar{2}), \text{nrW}(\bar{z}, 2), \text{1W}(x, 2)\}$$

and relations:



The set of initial events E_0 is not shown in the figure, but since these events have

only outgoing edges, they cannot be part of any cycle. T is RDMA-consistent, since **ob** is acyclic, and violating, since **sc** is cyclic. Furthermore, it is minimal, since removing the last event (w.r.t. **po**) makes it non-violating. Note that we cannot remove any other event, since this would not yield a valid trace of R .

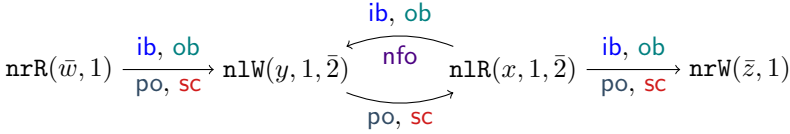
We choose the maximal event $e = 1W(x, 2)$ and consider one possible ordering of when events happen, called a *linearisation* $\tau = \tau_1 \ e \ \tau_2$:

$$\tau = \text{nrR}(\bar{w}, 1) \ 1W(x, 2) \ \text{n1R}(x, 2, \bar{2}) \ \text{nrW}(\bar{z}, 2) \ \text{n1W}(y, 1, \bar{2})$$

Removing e from τ and adjusting the values of events that depended on it yields the event sequence τ' :

$$\tau' = \text{nrR}(\bar{w}, 1) \ \text{n1R}(x, 1, \bar{2}) \ \text{nrW}(\bar{z}, 1) \ \text{n1W}(y, 1, \bar{2})$$

The corresponding trace T' on the event set $E' = E \setminus \{e\}$ is:



We observe that the values of the read and write events of the instruction $z := x$ have been adjusted, since they now read and write the initial value instead of the value written by the removed event. T' is both RDMA-consistent and SC-consistent, since $T'.\text{ob}$ and $T'.\text{sc}$ are acyclic, respectively.

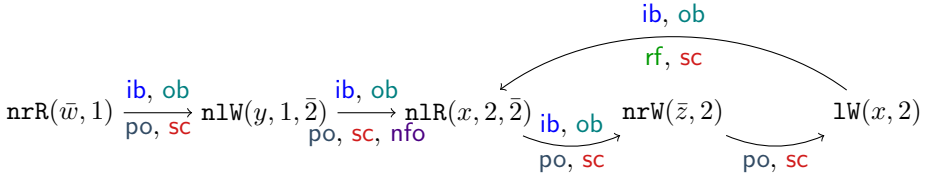
Since T' is SC-consistent, we can find an ordering of its events that respects all SC constraints:

$$\sigma = \text{nrR}(\bar{w}, 1) \ \text{n1W}(y, 1, \bar{2}) \ \text{n1R}(x, 1, \bar{2}) \ \text{nrW}(\bar{z}, 1)$$

We now construct the normal form trace T'' by inserting the maximal event e back into this SC-consistent ordering. Specifically, we place events that happened before e in τ (only $\text{nrR}(\bar{w}, 1)$ in this case), then e itself, then the remaining events in their SC-consistent order:

$$\tau'' = \text{nrR}(\bar{w}, 1) \ 1W(x, 2) \ \text{n1W}(y, 1, \bar{2}) \ \text{n1R}(x, 2, \bar{2}) \ \text{nrW}(\bar{z}, 2)$$

The linearisation τ'' corresponds to the trace T'' :



It is RDMA-consistent and uses the same set of events as T . Furthermore, it has the same SC relation as T and is therefore also violating.

Note that although T and T'' have exactly the same events, their ordering relations differ. In particular, the **nfo**-relation between the NIC local write $\text{n1W}(y, 1, \bar{2})$ and the NIC local read $\text{n1R}(x, 1, \bar{2})$ has been reversed. Intuitively, the normal form eliminates the unnecessary delay of $\text{n1W}(y, 1, \bar{2})$ after the read and write events of the later instruction $z := x$.

5.2 Definitions and Properties

Definition 7 (Linearisation of a Trace). A linearisation of an RDMA-consistent trace $T = \langle E, \text{po}, \text{pf}, \text{rf}, \text{mo}, \text{nfo} \rangle$ is a strict total order on the events $E = \{e_1, \dots, e_n\}$ that is an extension (a superset) of $T.\text{ob}$. We usually write it as a sequence $\tau = e_1 \dots e_n$. Similarly, a linearisation of an SC-consistent trace T is a strict total order extending $T.\text{sc}$.

Given a sequence of events $\tau = e_1 \dots e_n$, we write $e_i <_\tau e_j$ if $i < j$. Furthermore, we define $T(\tau) = \langle E, \text{po}, \text{pf}, \text{rf}, \text{mo}, \text{nfo} \rangle$, where:

- $E = \{e_1, \dots, e_n\}$
- po and pf are defined as in Definition 2 with respect to the event set E
- $e \xrightarrow{\text{rf}} e'$ if and only if $e <_\tau e'$, e is a write event, e' is a read event on the same variable, and there is no other write event on the same variable between e and e' in τ
- $e \xrightarrow{\text{mo}} e'$ if and only if $e <_\tau e'$ and both e and e' are write events on the same variable
- $e \xrightarrow{\text{nfo}} e'$ if and only if $e <_\tau e'$, both e and e' are events belonging to the same queue pair, exactly one of them is a read event and one of them is a write event, and they are either both local or both remote NIC events

$T(\tau)$ is a (not necessarily RDMA-consistent) trace of R if and only if the event set E , the polls-from relation pf and the reads-from relation rf are *well-formed*, meaning they are in the form as required in Definition 2. In particular, there must be at least as many NIC writes as polls on each queue pair at any time, and each pair of events related by rf must have matching values. All other requirements for relations are automatically fulfilled.

Definition 8 (Maximal Event). Given a trace T , an event $e \in E$ is maximal if it is po -maximal and there is no CPU-event e' such that $e \xrightarrow{\text{ib}} e'$.

Lemma 9. Any RDMA-consistent trace T has at least one maximal event e .

Proof. Assume that there is no maximal event. Construct a sequence of events $e_1, e'_1, e_2, e'_2, \dots$ as follows. Let e_1 be any po -maximal event. For all $i \in \mathbb{N}$, let e'_i be a CPU-event such that $e_i \xrightarrow{\text{ib}} e'_i$ and let e_{i+1} be a po -maximal event such that $e'_i \xrightarrow{\text{po}^*} e_{i+1}$. Because there are only finitely many po -maximal events, this sequence must eventually repeat and form a cycle. Since po -edges from CPU-events to later events are in ippo and therefore in ib , the cycle is completely contained in ib , which contradicts that T is RDMA-consistent.

5.3 Construction

Consider an RDMA program R that is not robust. Choose a violating trace T that is minimal in the sense that any trace with fewer events is not violating. In particular, $T = \langle E, \text{po}, \text{pf}, \text{rf}, \text{mo}, \text{nfo} \rangle$ is RDMA-consistent and $T.\text{sc} = \langle E, \text{po}, \text{rf}, \text{rb}, \text{mo} \rangle$

is cyclic. Let e be a maximal event and let $\tau = \tau_1 \ e \ \tau_2$ be a linearisation of T . We remove the maximal event e from τ to obtain τ' in the following way. For each event $\bar{e} \in E \setminus \{e\}$ in τ :

- If $\bar{e}$ is a NIC local read $\mathbf{nlR}(x, v_r, \bar{n})$ or a NIC remote read $\mathbf{nrR}(\bar{y}, v_r)$, we replace the value v_r with the value of the last write event on x or $\bar{y}$, respectively.
- If $\bar{e}$ is a NIC remote write $\mathbf{nrW}(\bar{y}, v_w)$ or a NIC local write $\mathbf{nlW}(x, v_w, \bar{n})$, we replace the value v_w with the value of the local or remote read coming immediately before $\bar{e}$ in $\mathbf{po}$.
- Otherwise we keep the event $\bar{e}$ unchanged.

We claim that $T' = T(\tau')$ is an RDMA-consistent trace of R . Since T was minimal, it follows that T' is not violating and therefore $T'.\mathbf{sc}$ is acyclic. Let σ be a linearisation of $T'.\mathbf{sc}$. Define $\tau'' = \tau_1 \downarrow_\sigma \ e \ \tau_2 \downarrow_\sigma$ where $\tau_i \downarrow_\sigma$ are the events of τ_i in the order in which they appear in σ .

Theorem 10. *$T'' = T(\tau'')$ is a violating trace of R .*

The full proof can be found in the extended version [9]. It proceeds in two phases, corresponding to the two trace transformations performed on the original trace T . In the first phase, we establish that after removing the maximal event e from τ and adjusting dependent NIC read/write values, the resulting trace T' remains both well-formed and RDMA-consistent. In the second phase, we analyse the trace T'' obtained by reinserting e into T' . The key structural observation is that several fundamental relations ($\mathbf{po}$, $\mathbf{ippo}$, $\mathbf{oppo}$, $\mathbf{pf}$, $\mathbf{rf}$, $\mathbf{mo}$, $\mathbf{rb}$) remain unchanged between T and T'' . Building on this, we prove that T'' is RDMA-consistent by showing all $\mathbf{ob}''$ -edges follow the order given by τ'' , that is, they relate events earlier in τ'' to events later in τ'' . Finally, we establish that $T.\mathbf{sc} = T''.\mathbf{sc}$, completing the proof that T'' violates the same SC properties as T while being RDMA-consistent.

This establishes Theorem 6: the partition required by the theorem is given by $E_1 = \{e\} \cup \{e' \in E'' \mid e' \in \tau_1\}$ and $E_2 = \{e' \in E'' \mid e' \in \tau_2\}$. The SC-consistency within each partition follows from the construction, which orders events in τ_1 and τ_2 according to the SC-linearisation σ of T' . Note that due to maximality, e is $\mathbf{po}$ -later than any event in τ_1 , which means it can be added to E_1 while preserving SC-consistency. The directionality of $\mathbf{ob}$ -edges between the two partitions is guaranteed by the proof that all $\mathbf{ob}''$ -edges respect the ordering of τ'' .

6 An EXPSPACE-Algorithm for Robustness

6.1 Overview

The previous section established that every non-robust RDMA program has a violating trace whose event set can be partitioned as $E = E_1 \cup E_2$ where events within each partition are SC-consistent and all $\mathbf{ob}$ -edges between partitions go

from E_1 to E_2 . This chapter leverages this normal form to establish an upper bound for the computational complexity of the robustness problem.

We show that RDMA-robustness can be reduced to reachability in a finite-state program augmented with net counters ($\in \mathbb{N}$), which implies membership in EXPSpace: This follows immediately from the fact that the state reachability problem for Vector Addition Systems with States (VASS) is in EXPSpace [26]. For RDMA programs that do not use poll operations, the reduction yields a purely finite-state program without counters. Since state reachability for finite-state programs is in PSPACE [28], we obtain PSPACE-membership for the special case.

Theorem 11 (Upper Bounds for RDMA-Robustness).

1. *RDMA-robustness is in EXPSpace in the general case.*
2. *RDMA-robustness for programs without poll operations is in PSPACE.*

The key to establishing the upper bounds is to construct an instrumented program running under SC that systematically searches for violating traces in normal form. The instrumented program builds up the two partitions E_1 and E_2 in parallel, non-deterministically assigning each generated event to either partition. To implement this effectively, we work with linearisations τ_1 and τ_2 (explicit total orderings of E_1 and E_2) rather than the partitions directly. This choice is motivated by the need to verify RDMA-consistency: maintaining explicit orderings allows us to check that all **ob**-edges are forward edges, which is simpler than reasoning about acyclicity in a partial order.

In the following, $\tau = \tau_1\tau_2$ denotes the violating trace in normal form that we are constructing. Events are appended to the end of either τ_1 or τ_2 , preserving SC-consistency within each sequence. The instrumented program ensures that the concatenation $\tau = \tau_1\tau_2$ forms an RDMA-consistent trace with an SC-cycle, thereby witnessing non-robustness of the original RDMA program.

6.2 Key Idea

The key idea is to simulate the program R and, for each generated event, non-deterministically decide whether it belongs to τ_1 or τ_2 . The instrumented program runs two simulations in parallel, building up τ_1 and τ_2 in the process: The first simulation tracks the effects of the events of τ_1 on the memory, while the second simulation does the same for the events of τ_2 on a separate copy of the memory.

Crucially, generated events are always appended to the end of either τ_1 or τ_2 . This preserves SC-consistency *within* each τ_i , but not necessarily *between* τ_1 and τ_2 . The instrumented program must keep track of and ensure RDMA-consistency of the resulting trace $\tau = \tau_1\tau_2$. Since we are trying to guess τ as an **ob**-linearisation, all **ob**-edges must be forward edges. If all edges are both well-formed and forward edges, then the resulting trace is RDMA-consistent.

Memory Consistency. The memory after simulating τ_1 must match the memory at the start of the simulation of τ_2 . To ensure this, we non-deterministically guess the memory state after τ_1 at the start of the program and use this for the simulation of τ_2 . After the instrumented program has simulated all of τ , it checks that the memory of the τ_1 -simulation matches what the τ_2 -simulation started with.

SC-Cycle Detection. The algorithm must also check whether there exists an SC-cycle. An SC-cycle consists of a cycle of events where each event and its successor are related by **po**, **rf**, **rb**, or **mo**. At a high level, we guess which threads participate in the cycle and in which order they are visited. During simulation, each thread may mark one entry and one exit event as its local contribution to the cycle and store the corresponding metadata (type, variable, value, and whether the event is in τ_1 or τ_2). After simulation stops, the main thread checks whether these local guesses compose into a valid global SC-cycle. The precise justification and checks are given in Sect. 6.5.

6.3 Algorithm Description

The instrumented program consists of one main thread that orchestrates the simulation plus one simulation thread for each thread in the original RDMA program. Additionally to some bookkeeping and status variables, the instrumented program stores the following:

- Three copies of each variable of the RDMA program: one copy to simulate τ_1 and τ_2 respectively, and the third to check that the value after τ_1 matches the memory at the beginning of τ_2 .
- For each thread, information about the entry and exit point of the SC cycle, together with the threads's predecessor and successor in the SC cycle.
- For each queue pair, information about which type of events have already been added to τ_2 .
- For each queue pair, two counters to track the number of polls that can be executed in τ_1 and τ_2 , respectively.

The instrumented program works as follows:

1. The main thread guesses the value of each variable after executing τ_1 . It keeps each value in two copies of the variable: one for the τ_2 -simulation to work on, and one to later verify that the value matches the result after τ_1 .
2. The main thread signals the other threads to start simulating.
3. Concurrently, each simulation thread of the instrumented program simulates a thread of the RDMA program as follows. For each simulated event:
 - (a) Non-deterministically decide whether the event should be in τ_1 or τ_2 .
 - (b) Check that the event can be added to τ_1 or τ_2 , respectively, without violating RDMA-consistency (see Sect. 6.4).
 - (c) Execute the effect of the event on the copy of the memory corresponding to τ_1 or τ_2 , respectively.

- (d) Non-deterministically decide whether the event should be the entry point of the SC-cycle in this thread. If so, save the event and check that it can form a valid connection (via **rf**, **mo**, or **rb**) with the corresponding exit point of its predecessor thread in the SC-cycle (see Sect. 6.5).
 - (e) Do the same for the exit point.
4. Non-deterministically, the main thread decides to signal the other threads to stop the simulation. It waits for each thread to finish their current event.
 5. The main thread checks that the memory after τ_1 matches the memory at the beginning of τ_2 .
 6. The main thread checks that there is a valid SC-cycle. If so, it enters the *target state*.

The original RDMA program is robust if and only if the instrumented SC program *cannot* reach the target state.

6.4 Ensuring RDMA-Consistency

To ensure that an event can be added to the trace in an RDMA-consistent way (step 3b), the program must check that each **ob**-edge is a forward edge. Additionally, it must verify that **rf** is well-formed, i.e., that the value of a read event matches the value of its write event.

The most challenging cases are **oppo** and $([\text{Inst}]; \text{ib})$, which can potentially form backward edges. For **oppo**, we track per queue pair which event types have been added to τ_2 , preventing events from being added to τ_1 if this would create a backward edge. For $([\text{Inst}]; \text{ib})$, we perform additional checks based on case analysis to ensure all paths remain forward-directed. The poll-from relation $([\text{n1w}]; \text{pf})$ requires counters to track unpolled NIC writes, with additional book-keeping to force certain polls into τ_2 . The remaining relations (**rf**, **nfo**, **rb**, **mo**) are always forward edges and require only basic well-formedness checks.

Details of all consistency checks can be found in the extended version [9].

6.5 Detecting SC-Cycles

To detect SC-cycles, we non-deterministically guess the entry and exit points of the SC-cycle in each thread. The entry and exit points are the **po**-first and **po**-last events of the thread on the cycle, respectively.

A key observation is that it is sufficient to consider cycles that visit each thread at most once. This is independent of the normal-form assumption (and of RDMA robustness): it holds for any cycle over **po**, **rf**, **rb**, and **mo** edges. If a cycle visits some thread more than once, let e_1 be the **po**-latest cycle event on that thread, and let e_2 be the next cycle event on the same thread encountered when following the cycle after e_1 . Then the segment from e_2 to e_1 can be contracted to the single edge $e_2 \xrightarrow{\text{po}} e_1$, yielding a shorter cycle. Repeating this contraction removes repeated visits, so an equivalent cycle exists that visits each thread at most once.

Therefore, it is enough to check connectivity between consecutive threads in the guessed cycle order by verifying that their selected exit/entry events are related by **rf**, **mo**, or **rb**.

The instrumented program ensures proper ordering between exit and entry points across partition boundaries (τ_1 and τ_2) through appropriate tracking during simulation. After all threads finish, the main thread verifies that all pairs of entry and exit points form a valid cycle.

Details can be found in the extended version [9].

6.6 Complexity Analysis

The instrumented program described above establishes the upper bounds of robustness as in Theorem 11. It uses a finite amount of state to track:

- Three copies of the memory (for τ_1 and τ_2 simulations and comparison),
- Per-queue-pair tracking of event types added to τ_2 (for **oppo** checking),
- Information about entry and exit points of the SC-cycle,
- Additional bookkeeping for **ib**-consistency checks.

All of this requires space polynomial in the size of the original RDMA program.

For programs with poll operations, the instrumented program additionally uses two counters per queue pair to track unpollled NIC writes. This makes the instrumented program a finite-state program augmented with net counters, which is essentially a vector addition system with states. Since the reachability problem for VASS is decidable in EXPSpace [26], we obtain EXPSpace-membership.

For programs without poll operations, no counters are needed, yielding a purely finite-state instrumented program. Reachability for finite-state programs is in PSPACE [28], thus robustness for poll-free programs is in PSPACE.

To validate this construction, we have implemented a tool that translates RDMA programs to instrumented Promela programs, which can then be verified using the SPIN model checker. SPIN is not intended for unbounded VASS-style counter reachability in full generality; however, it is sufficient for our proof-of-concept experiments on small acyclic litmus tests, where the number of polls is bounded and the translation therefore requires only bounded counters. We evaluated our tool on all sequential and concurrent litmus tests created by Ambal et al. [10], all of which produced the expected results.

7 Robustness is EXPSpace-Complete

The previous section established that RDMA-robustness is decidable in EXPSpace for the general case and in PSPACE for programs without poll operations. We now show that these upper bounds are tight by establishing matching lower bounds. This demonstrates that the algorithm from the previous section is optimal.

We establish EXPSPACE-hardness by reduction from the state-reachability problem for Vector Addition Systems with States (VASS) [22]. A VASS consists of a finite-state control with a finite number of counters ranging over the natural numbers $\mathbb{N}$. Each transition either increments or decrements a counter. Increment transitions are always enabled, while decrement transitions can only fire when the counter value is strictly positive. The state-reachability problem asks whether, starting from a given initial state with all counters set to zero, there exists a sequence of transitions leading to a designated accepting state (with arbitrary counter values). This problem is EXPSPACE-complete [24, 26].

The key insight is that RDMA programs can naturally simulate VASS executions: control states are encoded in program variables, while counter values correspond to the number of unpollled remote writes on queue pairs. We construct an RDMA program that simulates a VASS and then extend it to witness non-robustness precisely when the VASS reaches its accepting state.

7.1 Reduction from VASS

The reduction simulates a VASS execution using an RDMA program R . Given a VASS $V = (Q, d, T, q_0)$ with control states Q , d counters, transitions T , and initial state q_0 , we construct an RDMA program as follows. For each counter $c \in \{1, \dots, d\}$, we introduce an auxiliary node n_c containing a single variable x_c . A distinguished leader node n_L simulates the control state of the VASS using a local variable x .

The reduction maintains the following invariants: The leader node stores the current VASS control state in x . For each counter c , its value is represented by the number of remote writes to node n_c that have not yet been matched by a poll operation. Thus, issuing a remote write increments the counter, while a poll operation decrements it. Since each poll operation matches exactly one pending write, the number of pending acknowledgements corresponds precisely to the counter value.

More precisely, for each transition $t = (q, c, j, q') \in T$, the leader uses the code fragment given in Fig. 5 to simulate the transition t . It first checks that the value of x is q , then performs the RDMA operations corresponding to the value of j : an increment

of counter c (i.e., $j = 1$) is simulated by issuing a remote write to x_c , while a decrement (i.e., $j = -1$) is simulated by a poll operation on n_c .

Suppose the VASS has k transitions $\{t_1, t_2, \dots, t_k\}$. The leader repeatedly executes a non-deterministic choice among all such code fragments (one per transition), thereby simulating arbitrary VASS transition sequences. Formally, let C_i denote the code fragment corresponding to transition i . Let C_a be a code fragment that checks whether x equals the accepting state q_a . The leader node

1: <code>assume($x = q$)</code>	1: <code>assume($x = q$)</code>
2: <code>$\bar{x}_c := 1$</code>	2: <code>poll(n_c)</code>
3: <code>$x := q'$</code>	3: <code>$x := q'$</code>

Fig. 5. The RDMA code fragment corresponding to VASS transitions incrementing (left) and decrementing (right) a counter c .

executes the following loop: $(C_1 + C_2 + \dots + C_k + C_a)^*$. It follows that the accepting state q_a is reachable in the VASS if and only if C_a is reachable in R . The construction is polynomial in the size of the VASS. Note that R is robust: all operations execute in program order from the leader's perspective, and even though NIC writes may be delayed past subsequent polls, this is not observable since there are no competing reads or writes to the auxiliary nodes.

7.2 Extension to Robustness

To reduce to the robustness problem, we modify the RDMA program R into a new program R' such that R' is non-robust if and only if the state q_a is reachable in the VASS. We append some code fragment known to be non-robust (for example, the code from Sect. 5.1) to the end of C_a to obtain the new code C'_a . If needed, we also add additional nodes and variables to the RDMA program, as required by the non-robust code fragment. Let R' be the program $(C_1 + C_2 + \dots + C_k + C'_a)^*$.

If q_a is reachable in the VASS (i.e., C_a is reachable in R), then C'_a is reachable in R' , causing a robustness violation. Conversely, if q_a is not reachable in the VASS, then the code fragment C'_a can never be executed, and hence there is no robustness violation. This establishes EXPSpace-hardness for the general case.

For the restricted case of RDMA programs without poll operations, we use a simpler construction. Since no counters are needed, we reduce from the reachability problem for ordinary finite-state programs instead of VASS, which is PSPACE-hard [18].

Theorem 12 (Complexity of RDMA-Robustness).

1. *RDMA-robustness is EXPSpace-complete in the general case.*
2. *RDMA-robustness for programs without poll operations is PSPACE-complete.*

This establishes that the algorithm presented in the previous section is optimal: the EXPSpace upper bound cannot be improved without advances in the complexity of VASS reachability, and similarly for the PSPACE bound in the poll-free case. The robustness problem is thus computationally equivalent to reachability in finite-state programs with counters (general case) or without counters (poll-free case).

8 Conclusion

In this paper, we studied verification problems for RDMA programs and showed that reachability is undecidable in general, even for restricted fragments. We then proved that robustness for RDMA programs is decidable and EXPSpace-complete, based on a normal-form characterisation of robustness violations. This result yields a decision procedure that reduces robustness checking to reachability under sequential consistency, with a PSPACE bound for programs without poll operations. In contrast to prior RDMA robustness work that provided restrictive

sufficient criteria, our results give full decidability and complexity characterisations for robustness and for reachability-based safety verification under RDMA semantics.

We hope that our work will serve as a foundation for practical tools for automated robustness checking of RDMA programs. Our own implementation is a proof of concept, and scaling it to real-world benchmarks remains a challenge. We also plan to extend the techniques to richer RDMA features and to other baseline memory models.

Disclosure of Interests. The authors have no competing interests to declare.

Data-Availability Statement. Our prototype, along with the litmus tests used, is publicly available at <https://github.com/StephanSpengler/RDMA-implementation>.

References

1. Abdulla, P.A., Arora, J., Atig, M.F., Krishna, S.N.: Verification of programs under the release-acquire semantics. In: PLDI, pp. 1117–1132. ACM (2019). <https://doi.org/10.1145/3314221.3314649>
2. Abdulla, P.A., Atig, M.F., Bouajjani, A., Derevenetc, E., Leonardsson, C., Meyer, R.: On the state reachability problem for concurrent programs under power. In: Georgiou, C., Majumdar, R. (eds.) NETYS 2020. LNCS, vol. 12129, pp. 47–59. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-67087-0_4
3. Abdulla, P.A., Atig, M.F., Bouajjani, A., Kumar, K.N., Saivasan, P.: Deciding reachability under persistent x86-tso. *Proc. ACM Program. Lang.* **5**(POPL), 1–32 (2021). <https://doi.org/10.1145/3434337>
4. Abdulla, P.A., Atig, M.F., Bouajjani, A., Kumar, K.N., Saivasan, P.: Verification under intel-x86 with persistency. *Proc. ACM Program. Lang.* **8**(PLDI), 1189–1212 (2024). <https://doi.org/10.1145/3656425>
5. Abdulla, P.A., Atig, M.F., Bouajjani, A., Ngo, T.P.: The benefits of duality in verifying concurrent programs under TSO. In: CONCUR. LIPIcs, vol. 59, pp. 5:1–5:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2016). <https://doi.org/10.4230/LIPIcs.CONCUR.2016.5>
6. Abdulla, P.A., Atig, M.F., Godbole, A., Krishna, S., Vafeiadis, V.: The decidability of verification under PS 2.0. In: ESOP. Lecture Notes in Computer Science, vol. 12648, pp. 1–29. Springer, Cham (2021). <https://doi.org/10.26226/morressier.604907f41a80aac83ca25d26>
7. Abdulla, P.A., Atig, M.F., Kara, A., Rezone, O.: Verification of buffered dynamic register automata. In: Bouajjani, A., Fauconnier, H. (eds.) NETYS 2015. LNCS, vol. 9466, pp. 15–31. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-26850-7_2
8. Abdulla, P.A., Atig, M.F., Ngo, T.-P.: The best of both worlds: trading efficiency and optimality in fence insertion for TSO. In: Vitek, J. (ed.) ESOP 2015. LNCS, vol. 9032, pp. 308–332. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46669-8_13
9. Abdulla, P.A., Atig, M.F., Rajanbabu, G., Spengler, S.: On the verification problem of remote direct memory access programs (extended version with appendix) (2026). <https://arxiv.org/abs/2605.10631>

10. Ambal, G., Dongol, B., Eran, H., Klimis, V., Lahav, O., Raad, A.: Semantics of remote direct memory access: operational and declarative models of RDMA on TSO architectures. *Proc. ACM Program. Lang.* **8**(OOPSLA2), 1982–2009 (2024). <https://doi.org/10.1145/3689781>
11. Ambal, G., Lahav, O., Raad, A.: Sufficient conditions for robustness of RDMA programs. In: *ESOP* (1). *Lecture Notes in Computer Science*, vol. 15694, pp. 56–87. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-91118-7_3
12. Atig, M.F., Bouajjani, A., Burckhardt, S., Musuvathi, M.: On the verification problem for weak memory models. In: *POPL*, pp. 7–18. ACM (2010). <https://doi.org/10.1145/1706299.1706303>
13. Atig, M.F., Bouajjani, A., Burckhardt, S., Musuvathi, M.: What’s decidable about weak memory models? In: Seidl, H. (ed.) *ESOP 2012. LNCS*, vol. 7211, pp. 26–46. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-28869-2_2
14. Bouajjani, A., Derevenetc, E., Meyer, R.: Checking and enforcing robustness against TSO. In: Felleisen, M., Gardner, P. (eds.) *ESOP 2013. LNCS*, vol. 7792, pp. 533–553. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-37036-6_29
15. Bouajjani, A., Derevenetc, E., Meyer, R.: Robustness against relaxed memory models. In: *Software Engineering. LNI*, vol. P-227, pp. 85–86. GI (2014). <https://dl.gi.de/handle/20.500.12116/30973>
16. Bouajjani, A., Meyer, R., Möhlmann, E.: Deciding robustness against total store ordering. In: Aceto, L., Henzinger, M., Sgall, J. (eds.) *ICALP 2011. LNCS*, vol. 6756, pp. 428–440. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22012-8_34
17. Calin, G., Derevenetc, E., Majumdar, R., Meyer, R.: A theory of partitioned global address spaces. In: *FSTTCS. LIPIcs*, vol. 24, pp. 127–139. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2013). <https://doi.org/10.4230/LIPIcs.FSTTCS.2013.127>
18. Cheng, A., Esparza, J., Palsberg, J.: Complexity results for 1-safe nets. *Theor. Comput. Sci.* **147**(1&2), 117–136 (1995). [https://doi.org/10.1016/0304-3975\(94\)00231-7](https://doi.org/10.1016/0304-3975(94)00231-7)
19. Dan, A.M., Lam, P., Hoefer, T., Vechev, M.T.: Modeling and analysis of remote memory access programming. In: *OOPSLA*, pp. 129–144. ACM (2016). <https://doi.org/10.1145/2983990.2984033>
20. Derevenetc, E.: Robustness against Relaxed Memory Models. Ph.D. thesis, University of Kaiserslautern (2015). <https://nbn-resolving.org/urn:nbn:de:hbz:386-kluedo-40743>
21. Derevenetc, E., Meyer, R.: Robustness against power is PSpace-complete. In: Esparza, J., Fraigniaud, P., Husfeldt, T., Koutsoupias, E. (eds.) *ICALP 2014. LNCS*, vol. 8573, pp. 158–170. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-662-43951-7_14
22. Hopcroft, J.E., Pansiot, J.: On the reachability problem for 5-dimensional vector addition systems. *Theor. Comput. Sci.* **8**, 135–159 (1979). [https://doi.org/10.1016/0304-3975\(79\)90041-0](https://doi.org/10.1016/0304-3975(79)90041-0)
23. Lahav, O., Boker, U.: What’s decidable about causally consistent shared memory? *ACM Trans. Program. Lang. Syst.* **44**(2), 8:1–8:55 (2022). <https://doi.org/10.1145/3505273>
24. Lipton, R.J.: The reachability problem requires exponential space. Research report (Yale University. Department of Computer Science), Department of Computer Science, Yale University (1976). <https://books.google.de/books?id=7iSbGwAACAAJ>

25. Post, E.L.: A variant of a recursively unsolvable problem. *Bull. Am. Math. Soc.* **52**(4), 264–268 (1946). <https://doi.org/10.1090/S0002-9904-1946-08555-9>
26. Rackoff, C.: The covering and boundedness problems for vector addition systems. *Theor. Comput. Sci.* **6**, 223–231 (1978). [https://doi.org/10.1016/0304-3975\(78\)90036-1](https://doi.org/10.1016/0304-3975(78)90036-1)
27. Singh, A.K., Lahav, O.: Decidable verification under localized release-acquire concurrency. In: *TACAS (3)*. *Lecture Notes in Computer Science*, vol. 14572, pp. 235–254. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-57256-2_12
28. Sistla, A.P., Clarke, E.M.: The complexity of propositional linear temporal logics. *J. ACM* **32**(3), 733–749 (1985). <https://doi.org/10.1145/3828.3837>

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

